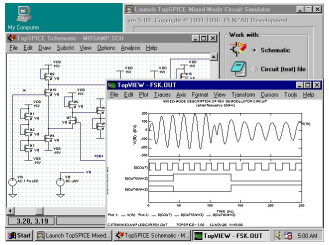
컴퓨터 시스템



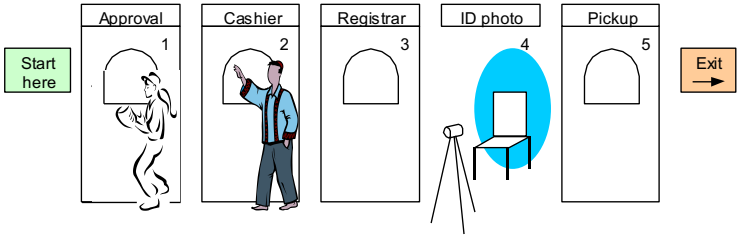
Parallelism

Pipelining Concepts

Strategies for improving performance

1 – Use multiple independent data paths accepting several instructions that are read out at once: *multiple-instruction-issue* or *superscalar*

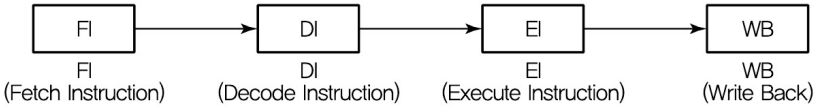
2 – Overlap execution of several instructions, starting the next instruction before the previous one has run to completion: *(super)pipelined*



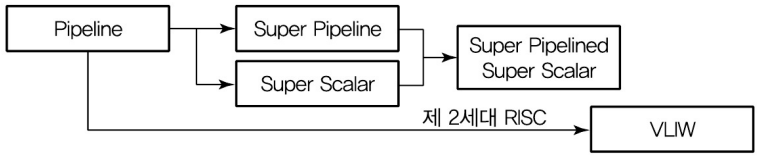
Pipelining in the student registration process.

파이프라인

- 명령어 처리 과정을 여러 단계로 세분화해서 동시에 서로 다른 작업들이 수행되도록 하여 병렬성을 높이는 기법
- CPU의 프로그램 처리 속도를 높이기 위하여 CPU 내부 하드웨어를 여러 단계로 나누어 동시에 처리하는 기술
- 수행과정



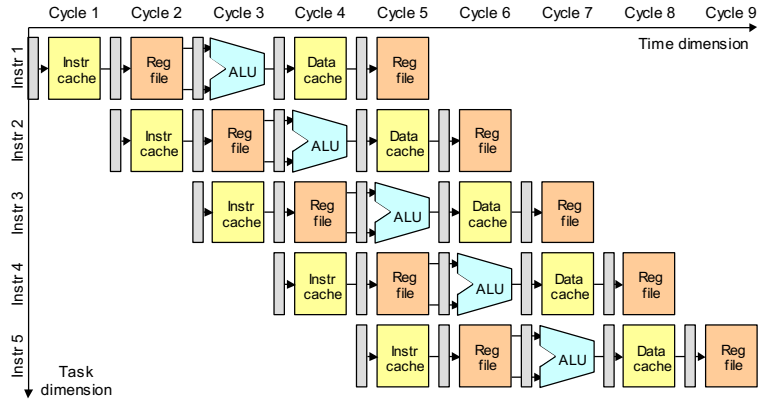
- 파이프라인 유형



단일 파이프라인

- 효과적인 병렬처리를 위해 몇 가지 동작을 명령어 수행과정에서 각 단계를 한번만 중첩하는 기술

Pipelined Instruction Execution



Pipelining in the MicroMIPS instruction execution process.

Embedded Computing Laboratory
School of Information & Communication

파이프라인 사용시 성능향상

	1	2	3	4	5	6	7	8	9	10	→시간
1	F	D	E	W							
2		F	D	E	W						
3			F	D	E	W					
4				F	D	E	W				
5					F	D	E	W			
6						F	D	E	W		
7							F	D	E	W	

Embedded Computing Laboratory
School of Information & Communication

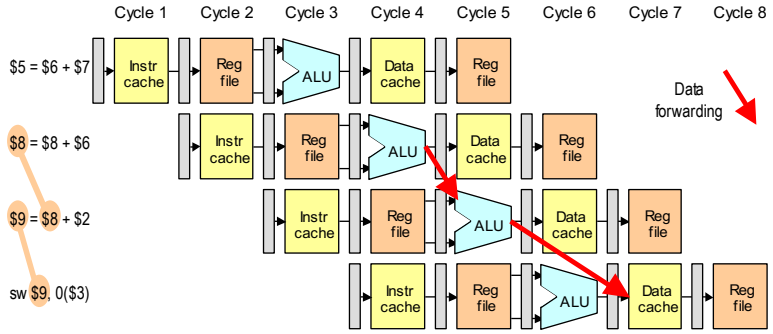
파이프라인의 문제점

- 파이프라인의 근본적 문제점
  - 모든 명령어들이 파이프라인 단계를 모두 거치지는 않는다
  - 어떤 명령어는 오퍼랜드를 인출할 필요가 없으나 구조를 단순화하기 위하여 4단계의 파이프라인을 통과
  - 클럭 스피드는 가장 긴 파이프라인 단계를 기준으로 설정
- 구조적 문제점
  - FI단계와 EI 단계가 동시에 기억장치를 사용할 경우 기억장치 충돌에 따른 지연시간
- 데이터 문제점
  - 이전 명령어의 실행 결과를 다음 명령어가 사용할 때 생기는 데이터 의존성에 따른 문제
- 제어 문제점
  - 분기(특히 조건분기)의 경우 미리 인출한 명령어를 사용할 수 없는 문제

Embedded Computing Laboratory
School of Information & Communication

Pipeline Stalls or Bubbles

First type of data dependency



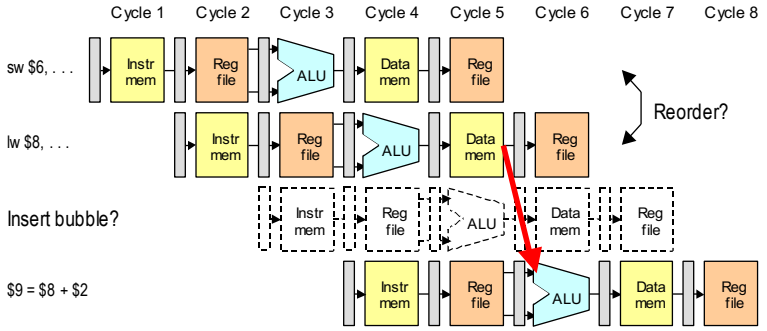
Read-after-write data dependency and its possible resolution through data forwarding .

Embedded Computing Laboratory
School of Information & Communication

파이프 라인의 문제점

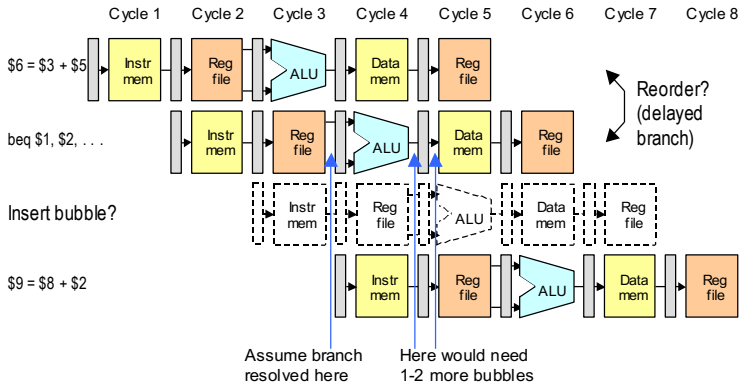
구조적 문제점	• 파이프라인을 위한 명령어 설계 파이프라인 단계들을 더욱 작게 분할함으로써 처리시간 차이를 최소화시켜 주는 슈퍼파이프라이닝 기술을 사용함
데이터 문제점	• 전방전달 파이프라인의 FI와 EI 단계가 직접 액세스하는 CPU 내부 캐시를 명령어 캐시와 데이터 캐시로 분리시키는 방법(별도의 H/W 추가) • 지연(Stall) 전방전달방식으로 해결하지 못하는 경우 해결될 때까지 지연시키는 방법
제어 문제점 (분기 문제점)	• 분기예측 분기가 일어날 것인지를 예측하고 그에 따라 어느 경로의 명령어를 인출할지를 결정하는 확률적 방법이다. 최근의 분기 결과들을 저장하여 두는 분기 역사표(branch history table)를 참조하여 예측하는 방법 • 분기목적지 선인출 조건 분기가 인식되면 분기명령어의 다음 명령어뿐 아니라 조건이 만족될 경우에 분기하게 될 목적지의 명령어도 함께 인출하는 방법 • 루프버퍼 사용 이 버퍼는 파이프라인의 명령어 인출 단계에 포함되어 있는 작은 고속 기억장치인데 가장 최근에 인출된 일정 개수의 명령어들이 순서대로 저장되어 있다. 이를 활용하여 프로그램을 처리하는 방식 • 지연분기 프로그램 내의 명령어들을 재배치함으로써 파이프라인의 성능을 개선하는 방법. 즉 분기 명령어의 위치를 적절히 조정하여 원래보다 나중에 실행되도록 재배치함으로써 그로 인한 성능 저하를 최소화함

Second Type of Data Dependency



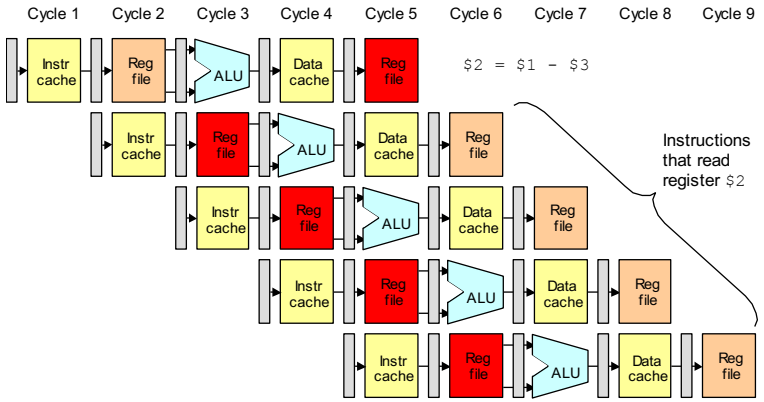
Read-after-load data dependency and its possible resolution through bubble insertion and data forwarding.

Control Dependency in a Pipeline



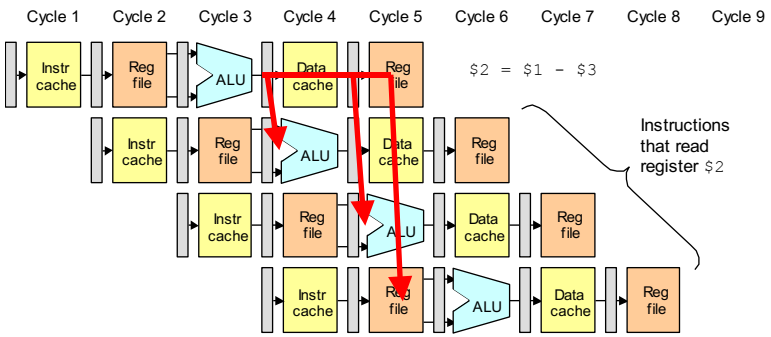
Control dependency due to conditional branch.

Data Dependencies and Hazards



Data dependency in a pipeline.

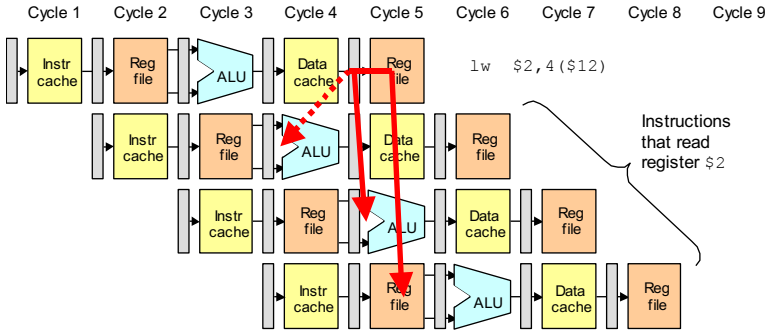
Resolving Data Dependencies via Forwarding



When a previous instruction writes back a value computed by the ALU into a register, the data dependency can always be resolved through forwarding.

Embedded Computing Laboratory
School of Information & Communication

Certain Data Dependencies Lead to Bubbles



When the immediately preceding instruction writes a value read out from the data memory into a register, the data dependency cannot be resolved through forwarding (i.e., we cannot go back in time) and a bubble must be inserted in the pipeline.

Embedded Computing Laboratory
School of Information & Communication

Pipeline Branch Hazards

Software-based solutions

Compiler inserts a “no-op” after every branch (simple, but wasteful)

Branch is redefined to take effect after the instruction that follows it

Branch delay slot(s) are filled with useful instructions via reordering

Hardware-based solutions

Mechanism similar to data hazard detector to flush the pipeline

Constitutes a rudimentary form of branch prediction:
Always predict that the branch is not taken, flush if mistaken

More elaborate branch prediction strategies possible

Embedded Computing Laboratory
School of Information & Communication

Branch Prediction

Predicting whether a branch will be taken

- Always predict that the branch will not be taken
- Use program context to decide (backward branch is likely taken, forward branch is likely not taken)
- Allow programmer or compiler to supply clues
- Decide based on past history (maintain a small history table); to be discussed later
- Apply a combination of factors: modern processors use elaborate techniques due to deep pipelines

Embedded Computing Laboratory
School of Information & Communication

A Simple Branch Prediction Algorithm

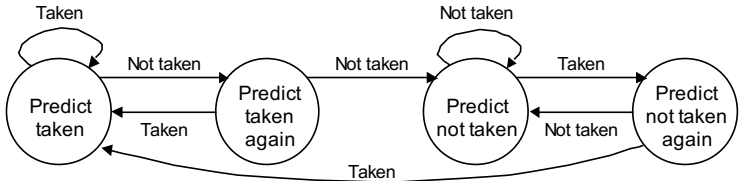


Fig. 16.6 Four-state branch prediction scheme.

Example 16.1

```
L1: ----          <-- 10 iter's
    ----
L2: ----          <-- 20 iter's
    ----
    br <c2> L2
    ----
    br <c1> L1
```

Impact of different branch prediction schemes

Solution

Always taken: 11 mispredictions, 94.8% accurate
1-bit history: 20 mispredictions, 90.5% accurate
2-bit history: Same as always taken

Embedded Computing Laboratory
School of Information & Communication

Hardware Implementation of Branch Prediction

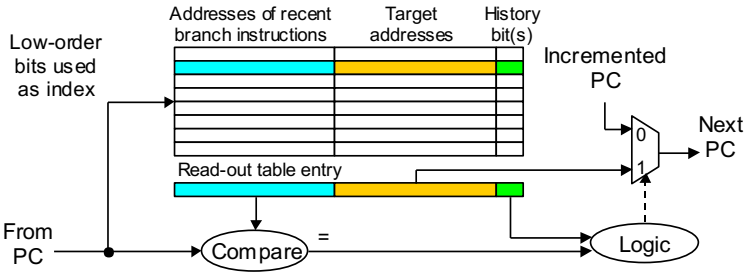


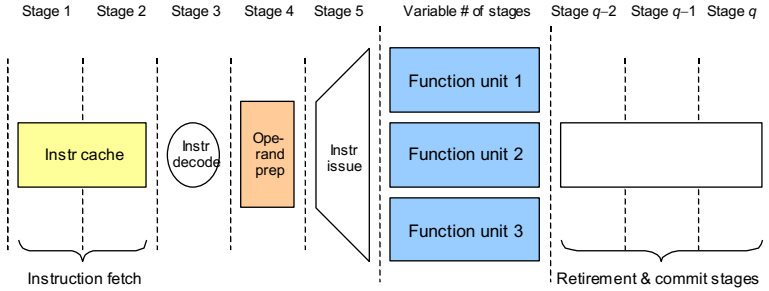
Fig. 16.7 Hardware elements for a branch prediction scheme.

The mapping scheme used to go from PC contents to a table entry is the same as that used in direct-mapped caches (Chapter 18)

Embedded Computing Laboratory
School of Information & Communication

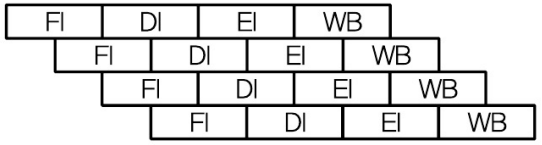
Advanced Pipelining

Deep pipeline = *superpipeline*; also, *superpipelined*, *superpipelining*
Parallel instruction issue = *superscalar*, *j-way issue* (2-4 is typical)



Embedded Computing Laboratory
School of Information & Communication

슈퍼 파이프라인

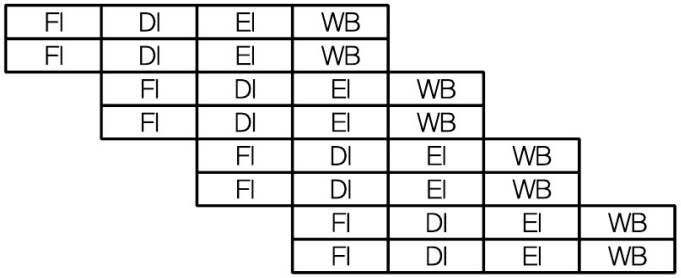


PIpelIning 더욱 세분화하여 수행시간을 단축

- 하나의 Pipeline을 여러 부분으로 나누어 연속적인 흐름과정으로 처리함으로써 성능을 향상하는 병렬처리 기술
- 단계를 세분화(1/2, 1/3 ——), Clock Cycle Time을 줄임
- 슈퍼스칼라보다 단순하고, 적은 부품을 사용
- Clock 수가 높아지면(Cycle Time이 짧아지면) 단계를 나누기가 어려움

Embedded Computing Laboratory
School of Information & Communication

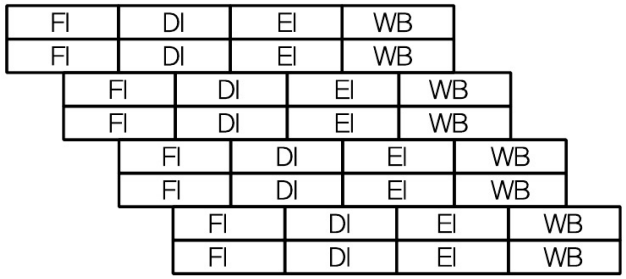
슈퍼스칼라



- 프로세서 내에 Pipeline 된 ALU를 여러 개 포함시켜서 매 사이클마다 복수의 명령어들이 동시에 실행될 수 있도록 하는 병렬처리 기술
- CPU 내에 파이프라인을 여러 개 두어 여러 명령어를 동시에 실행하는 기술
- 매 사이클마다 여러 명령을 동시에 수행
- H/W 수준 처리, Dynamic Pipeline Scheduling을 포함하도록 확장

Embedded Computing Laboratory
School of Information & Communication

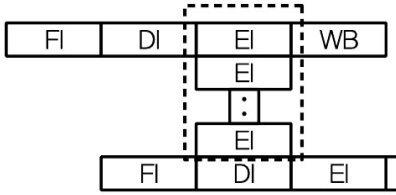
슈퍼파이프라인-슈퍼스칼라



- 슈퍼스칼라 기법에 슈퍼 파이프라이닝 기법을 적용하여 수행시간을 더 단축한 기법

Embedded Computing Laboratory
School of Information & Communication

VLIW(Very Long Instruction Word)



실행 Cycle이 여러 개의 기능 Unit으로 나누어져 동시에 처리됨

- 동시에 수행될 수 있는 명령어들을 컴파일러 수준에서 추출하여 하나의 명령어로 압축하여 수행하는 병렬처리 기술
- EPIC 기법 : 컴파일러가 소스 코드로부터 명시적 병렬성을 찾아 병렬처리가 가능하도록 기계어 코드 생성, 병렬 수행됨
- 정적다중처리, 인출/해독은 하나의 회로에서 처리, 실행만 여러 ALU에서 수행

Embedded Computing Laboratory
School of Information & Communication

Performance Improvement for Deep Pipelines

Hardware-based methods

Lookahead past an instruction that will/may stall in the pipeline
(out-of-order execution; requires in-order retirement)
Issue multiple instructions (requires more ports on register file)
Eliminate false data dependencies via register renaming
Predict branch outcomes more accurately, or speculate

Software-based method

Pipeline-aware compilation
Loop unrolling to reduce the number of branches

```
Loop:   Compute with index i          Loop:   Compute with index i
        Increment i by 1                      Compute with index i + 1
        Go to Loop if not done                Increment i by 2
                                              Go to Loop if not done
```


Embedded Computing Laboratory
School of Information & Communication

Superscalar

Term first coined in 1987

Refers to a machine that is designed to improve the performance of the execution of scalar instructions

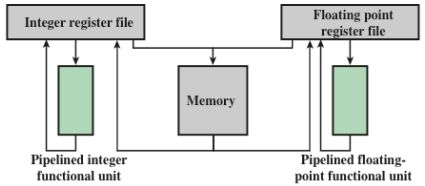
In most applications the bulk of the operations are on scalar quantities

Represents the next step in the evolution of high-performance general-purpose processors

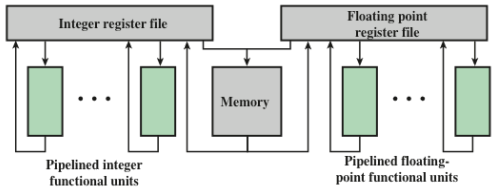
Essence of the approach is the ability to execute instructions independently and concurrently in different pipelines

Concept can be further exploited by allowing instructions to be executed in an order different from the program order

Superscalar vs. Ordinary Scalar



(a) Scalar organization



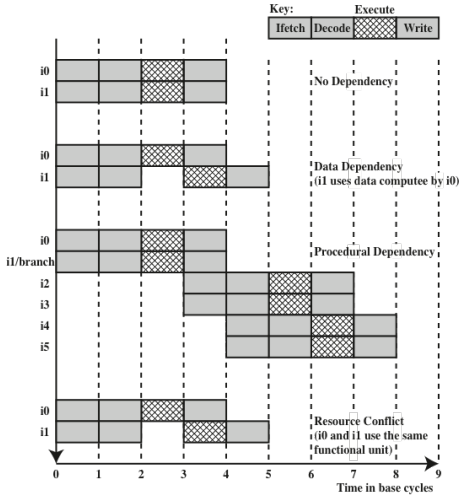
(b) Superscalar organization

Constraints

- Instruction level parallelism
  - Refers to the degree to which the instructions of a program can be executed in parallel
  - A combination of compiler based optimization and hardware techniques can be used to maximize instruction level parallelism
- Limitations:
  - True data dependency
  - Procedural dependency
  - Resource conflicts
  - Output dependency (WAW: Write after Write)
  - Antidependency(WAR: Write after Read)

```
B = 3
A = B + 1
B = 7
```


Effect of Dependencies



Branch Prediction

- Any high-performance pipelined machine must address the issue of dealing with branches
- Intel 80486 addressed the problem by fetching both the next sequential instruction after a branch and speculatively fetching the branch target instruction
- RISC machines:
  - Delayed branch strategy was explored
  - Processor always executes the single instruction that immediately follows the branch
  - Keeps the pipeline full while the processor fetches a new instruction stream
- Superscalar machines:
  - Delayed branch strategy has less appeal
  - Have returned to pre-RISC techniques of branch prediction

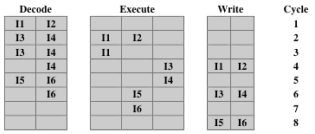
Design Issues – Instruction vs. Machine level

- Instruction level parallelism
  - Instructions in a sequence are independent
  - Execution can be overlapped
  - Governed by data and procedural dependency
- Machine Parallelism
  - Ability to take advantage of instruction level parallelism
  - Governed by number of parallel pipelines

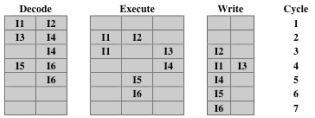
Instruction Issue Policy

- Instruction issue
  - Refers to the process of initiating instruction execution in the processor's functional units
- Instruction issue policy
  - Refers to the protocol used to issue instructions
  - Instruction issue occurs when instruction moves from the decode stage of the pipeline to the first execute stage of the pipeline
- Three types of orderings are important:
  - The order in which instructions are fetched
  - The order in which instructions are executed
  - The order in which instructions update the contents of register and memory locations
- Superscalar instruction issue policies can be grouped into the following categories:
  - In-order issue with in-order completion
  - In-order issue with out-of-order completion
  - Out-of-order issue with out-of-order completion

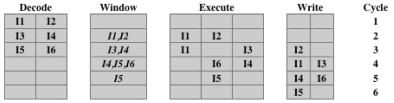
Superscalar Instruction Issue and Completion Policies



(a) In-order issue and in-order completion

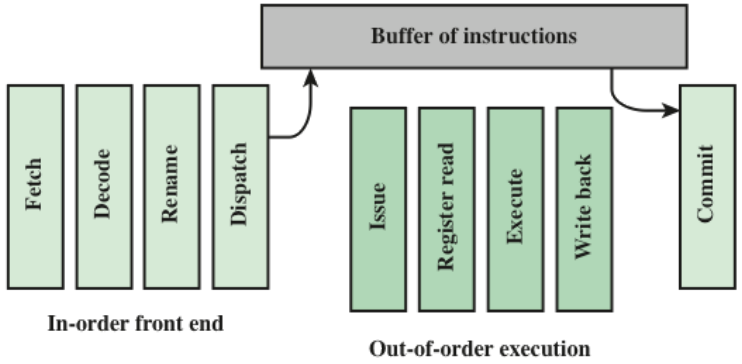


(b) In-order issue and out-of-order completion



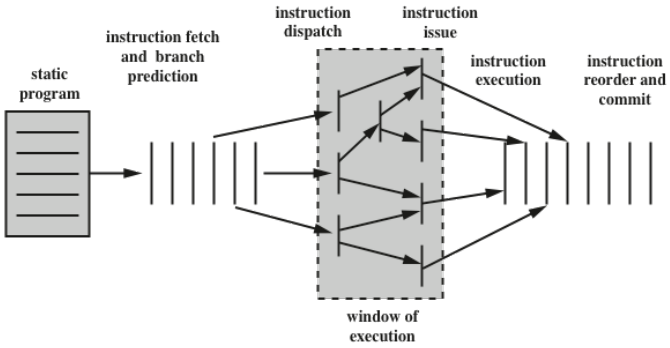
(c) Out-of-order issue and out-of-order completion

Organization for Out-of-Order Issue



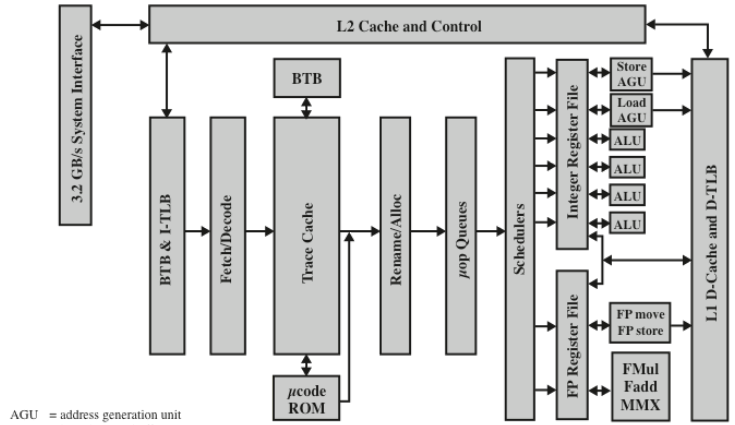
Embedded Computing Laboratory
School of Information & Communication

Conceptual Depiction of Superscalar Processing



Embedded Computing Laboratory
School of Information & Communication

Pentium 4 Block Diagram



AGU = address generation unit
BTB = branch target buffer
D-TLB= data translation lookaside buffer
I-TLB = instruction translation lookaside buffer

Embedded Computing Laboratory
School of Information & Communication

Pipeline Throughput with Dependencies

Assume that one bubble must be inserted due to read-after-load dependency and after a branch when its delay slot cannot be filled. Let β be the fraction of all instructions that are followed by a bubble.

$$\text{Pipeline speedup} = \frac{q}{(1 + q\tau/t)(1 + \beta)}$$

R-type	44%
Load	24%
Store	12%
Branch	18%
Jump	2%

Example 15.3

Calculate the effective CPI for MicroMIPS, assuming that a quarter of branch and load instructions are followed by bubbles.

Solution

Fraction of bubbles $\beta = 0.25(0.24 + 0.18) = 0.105$
CPI = $1 + \beta = 1.105$ (which is very close to the ideal value of 1)

Embedded Computing Laboratory
School of Information & Communication